

Final Exam (part 2) - Computational Physics I

Deadline: Thursday 5 June 2025 (by 23h59)

Credits: 10 points

9.5/10

Good work!

Please keep the structure provided below and submit an organised notebook with clear answers to each item.

Name: Juan Daniel Vasconez

2. Monte Carlo simulations: Cosmology (10 points)

Supernovae type Ia (SN Ia) are very energetic astronomical explosions, which have a very similar intrinsic known brightness (i.e. they have a very similar absolute magnitude M), so they can be used as cosmological "standard candles" to measure the luminosity distance, d , as a function of redshift, z :

$$d = \frac{cz}{H_0} \quad (1)$$

where c is the speed of light and H_0 is the Hubble constant. Since they have similar absolute magnitudes M , we can estimate distances by comparing how bright or faint they appear on the sky as indicated by the measured apparent magnitude, m , which does differ:

$$m = M + 5 \log \left(\frac{d}{\text{Mpc}} \right) + 25 \quad (2)$$

Higher m values imply objects are fainter; lower m values imply objects are brighter. Same for M . Unfortunately, selection effects associated with instrumental limitations can bias our measurements. For example, far-away SN Ia can be so faint that they may not be detectable, so the sample will be biased towards brighter objects. In the equation above Mpc stands for Mega parsecs, which are distance units used in cosmology.

Therefore, to understand selection bias, we want to simulate this effect using a Monte Carlo simulation.

The purpose of this problem is to determine the bias as a function of redshift for a sample of objects (SN Ia) via a Monte Carlo calculation. To set up your simulation, assume that:

- $H_0 = 70 \text{ km s}^{-1} \text{ Mpc}^{-1}$
- the absolute magnitude of SN Ia $M = -19.5 \text{ mag}$.
- your supernova search will be able to detect 100% of objects as faint as $m = 18.5 \text{ mag}$, and none fainter.

In [2]:

```
import numpy as np
import matplotlib.pyplot as plt
import scipy.constants as sc
import seaborn as sns
```

(a) Write a python function to generate N Gaussian random variables with mean $\langle M \rangle = -19.5 \text{ mag}$ and different standard deviations ($\sigma_M = 0.1, 0.3$, and 0.6 mag).

In [3]:

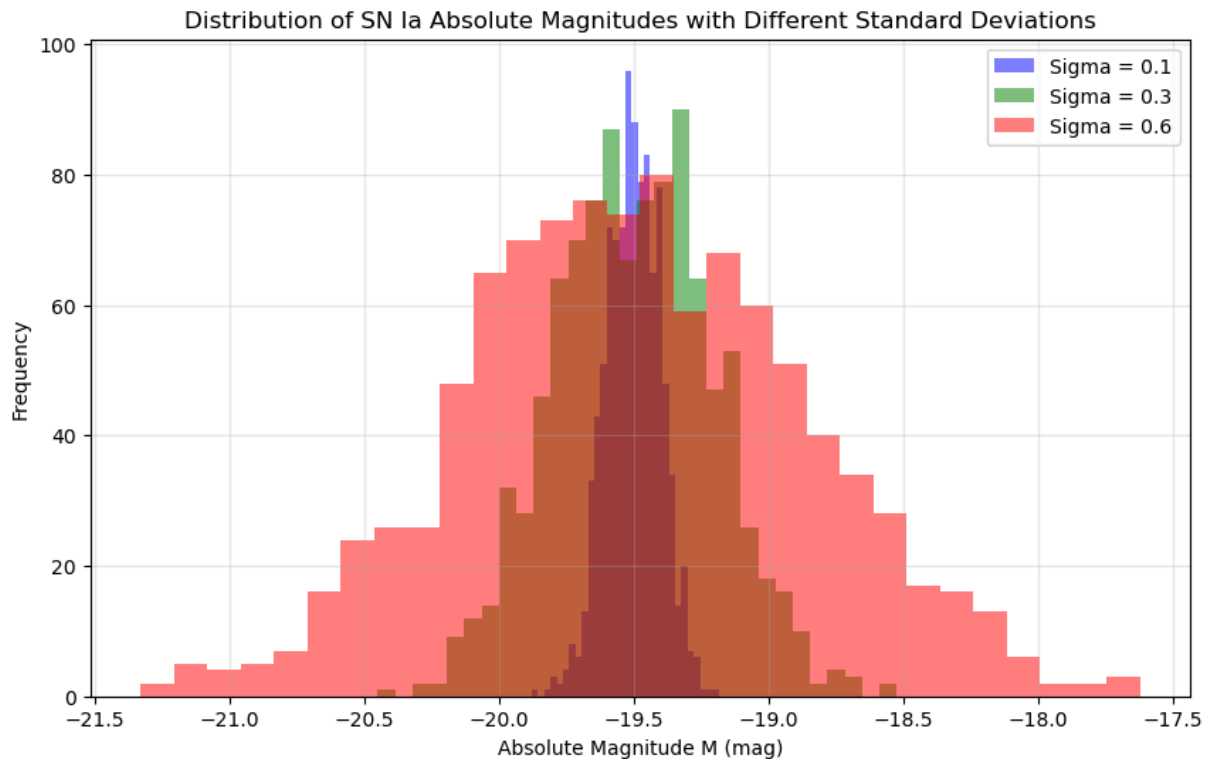
```
def random_gen(mean, stddev, n):
    """
    Generate a list of n random numbers with a given mean and standard deviation.

    :param mean: The mean of the random numbers.
    :param stddev: The standard deviation of the random numbers.
    :param n: The number of random numbers to generate.
    :return: A list of n random numbers.
    """
    import random
    return np.array([random.gauss(mean, stddev) for _ in range(n)])

# Define parameters for SN Ia absolute magnitude simulation
sigmas = [0.1, 0.3, 0.6]
mean = -19.5
n = 1000

# Generate Gaussian random variables for absolute magnitudes with different
sigma_1 = random_gen(mean, sigmas[0], n)
sigma_2 = random_gen(mean, sigmas[1], n)
sigma_3 = random_gen(mean, sigmas[2], n)

# Create histogram plot to visualize the distributions
plt.figure(figsize=(10, 6))
plt.hist(sigma_1, bins=30, alpha=0.5, label=f'Sigma = {sigmas[0]}', color='b')
plt.hist(sigma_2, bins=30, alpha=0.5, label=f'Sigma = {sigmas[1]}', color='g')
plt.hist(sigma_3, bins=30, alpha=0.5, label=f'Sigma = {sigmas[2]}', color='r')
plt.title('Distribution of SN Ia Absolute Magnitudes with Different Standard Deviations')
plt.xlabel('Absolute Magnitude M (mag)')
plt.ylabel('Frequency')
plt.legend()
plt.grid(True, alpha=0.3)
plt.show()
```



(b) Make 3 plots of M versus N , where N is the number of generated objects, one for each σ_M .

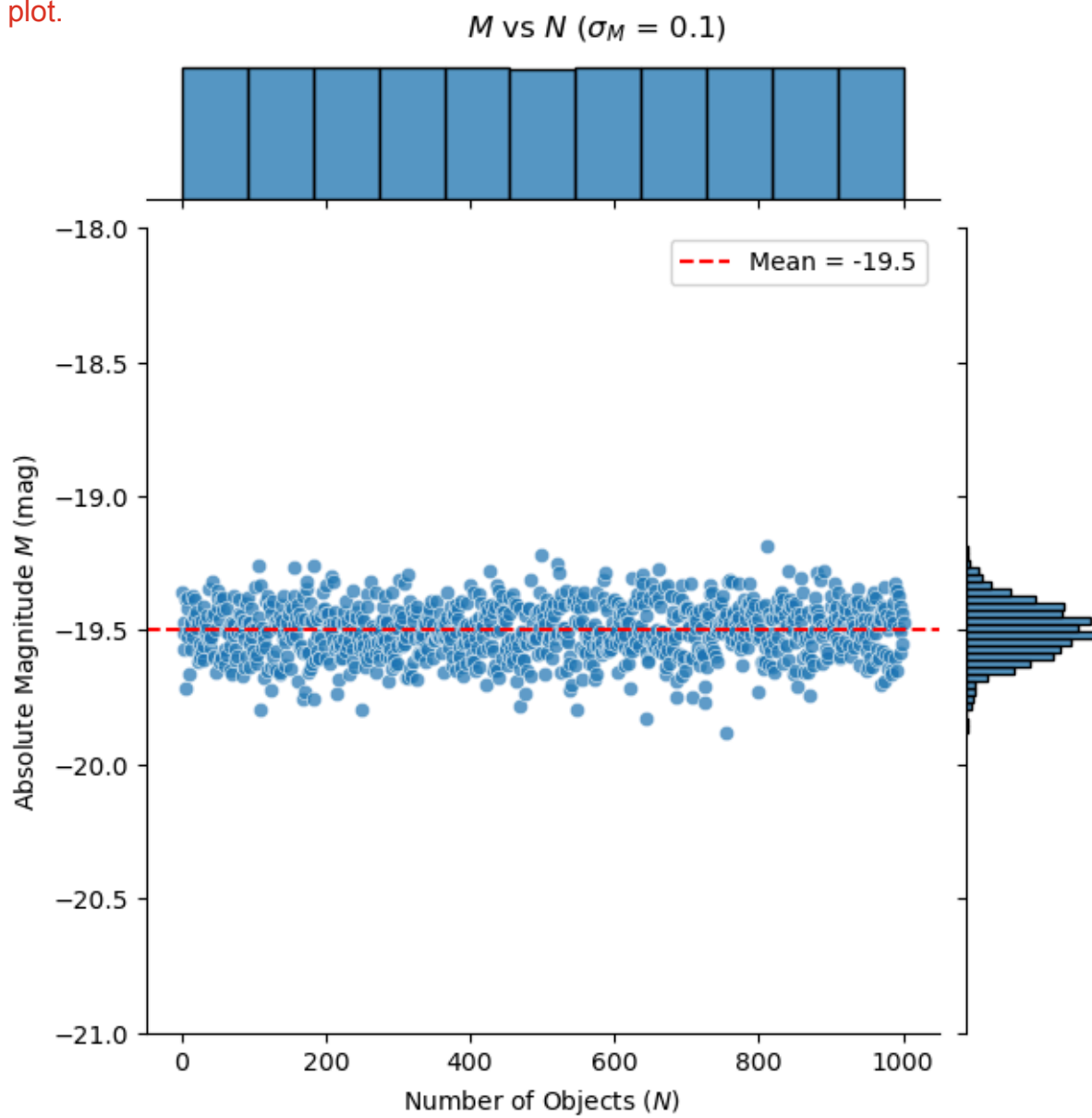
```
In [4]: # Create joint plots for each sigma
g1 = sns.jointplot(x=range(1, n+1), y=sigma_1, kind='scatter', alpha=0.7, height=10)
g1.figure.suptitle(f'$M$ vs $N$ ($\sigma_M$ = {sigmas[0]}'), y=1.02)
g1.set_axis_labels('Number of Objects ($N$)', 'Absolute Magnitude $M$ (mag)')
g1.ax_joint.axhline(y=mean, color='red', linestyle='--', label=f'Mean = {mean}')
g1.ax_joint.set_ylim(-21, -18)
g1.ax_joint.legend()
plt.show()

g2 = sns.jointplot(x=range(1, n+1), y=sigma_2, kind='scatter', alpha=0.7, height=10)
g2.figure.suptitle(f'$M$ vs $N$ ($\sigma_M$ = {sigmas[1]}'), y=1.02)
g2.set_axis_labels('Number of Objects ($N$)', 'Absolute Magnitude $M$ (mag)')
g2.ax_joint.axhline(y=mean, color='red', linestyle='--', label=f'Mean = {mean}')
g2.ax_joint.set_ylim(-21, -18)
g2.ax_joint.legend()
plt.show()

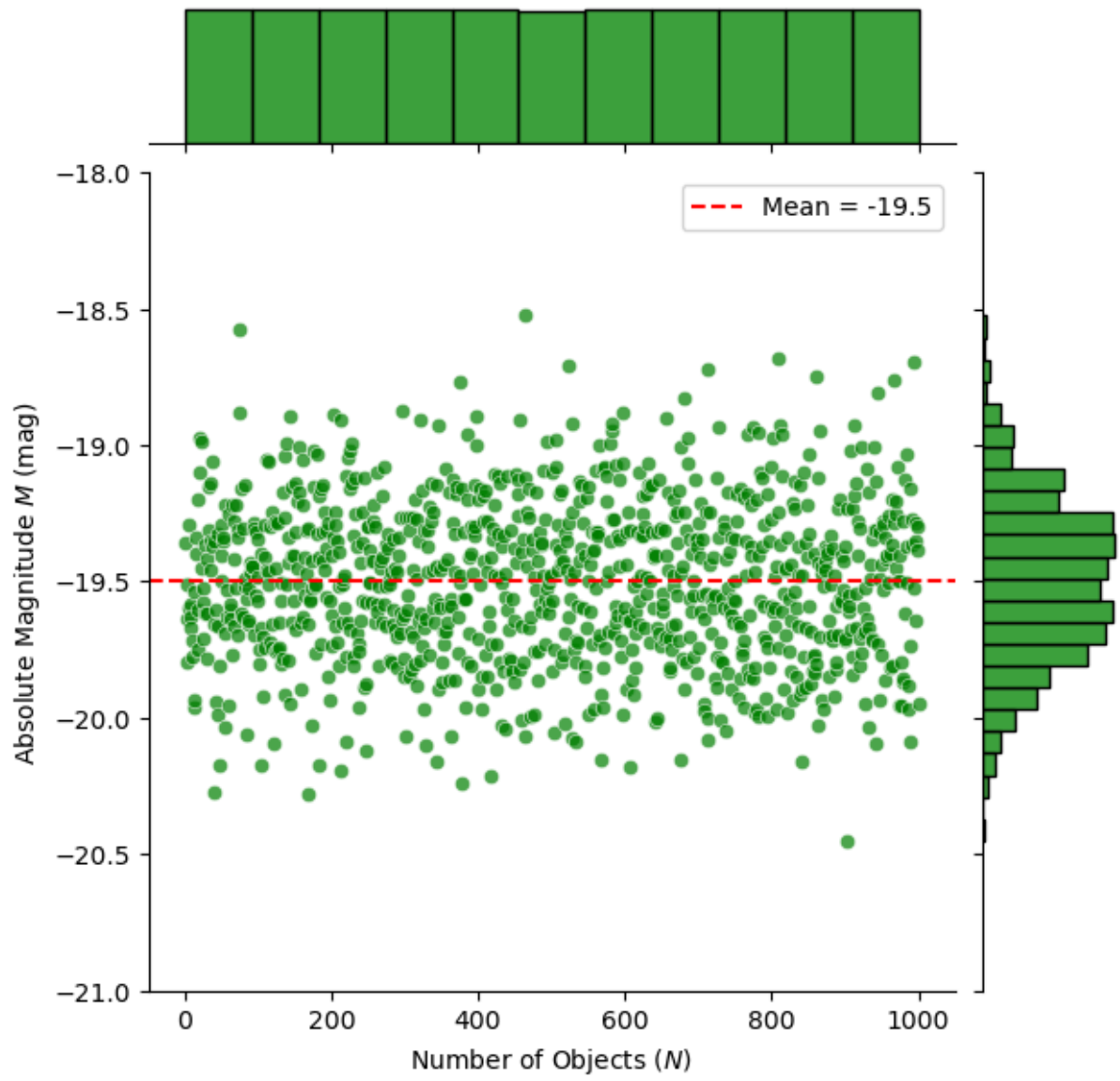
g3 = sns.jointplot(x=range(1, n+1), y=sigma_3, kind='scatter', alpha=0.7, height=10)
g3.figure.suptitle(f'$M$ vs $N$ ($\sigma_M$ = {sigmas[2]}'), y=1.02)
g3.set_axis_labels('Number of Objects ($N$)', 'Absolute Magnitude $M$ (mag)')
g3.ax_joint.axhline(y=mean, color='red', linestyle='--', label=f'Mean = {mean}')
g3.ax_joint.set_ylim(-21, -18)
g3.ax_joint.legend()
plt.show()
```

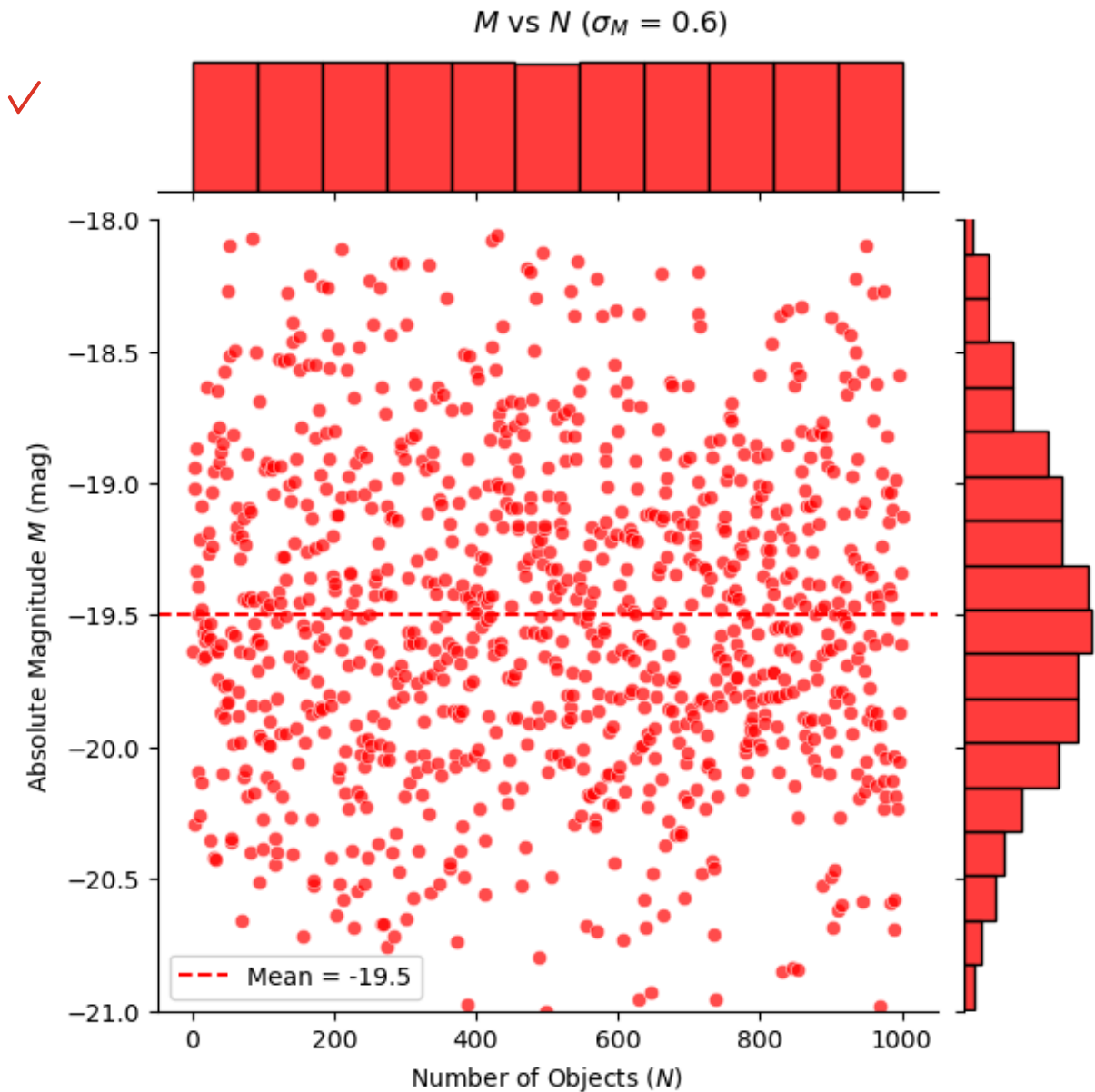
Very nice plot.

✓



M vs N ($\sigma_M = 0.3$)





(c) Write a python function to calculate and return:

- the luminosity distances, d , in Mpc given redshifts between $z = 0$ and $z = 0.1$.
- the apparent magnitudes, m , for the same redshift range.

In [5]:

```
def distances_and_magnitudes(z_values, M_values, H0=70):
    """
    Calculate luminosity distances and apparent magnitudes for given redshift

    Parameters:
    z_values: array of redshift values
    M_values: array of absolute magnitude values
    H0: Hubble constant in km/s/Mpc (default: 70)

    Returns:
    d: luminosity distances in Mpc
    m: apparent magnitudes
```

```

      """

      # Speed of light in km/s
      c = sc.c / 1000 # Convert from m/s to km/s

      # Calculate luminosity distances using d = cz/H0
      d = c * z_values / H0

      # Calculate apparent magnitudes using m = M + 5*log10(d/Mpc) + 25
      # Handle the case where d is zero or very small to avoid log(0)
      # Use a small minimum distance to avoid division by zero
      d_safe = np.maximum(d, 1e-10) # Set minimum distance to avoid log(0)
      m = M_values + 5 * np.log10(d_safe) + 25

      return d, m

# Create redshift range from 0 to 0.1
z_range = np.linspace(0, 0.1, 1000)

# Calculate distances and apparent magnitudes for each sigma case
d1, m1 = distances_and_magnitudes(z_range, sigma_1)
d2, m2 = distances_and_magnitudes(z_range, sigma_2)
d3, m3 = distances_and_magnitudes(z_range, sigma_3)

print(f"Distance range: {d1.min():.1f} to {d1.max():.1f} Mpc")
print(f"Apparent magnitude range for sigma=0.1: {m1.min():.2f} to {m1.max():.2f}")
print(f"Apparent magnitude range for sigma=0.3: {m2.min():.2f} to {m2.max():.2f}")
print(f"Apparent magnitude range for sigma=0.6: {m3.min():.2f} to {m3.max():.2f}")

```

Distance range: 0.0 to 428.3 Mpc
 Apparent magnitude range for sigma=0.1: -44.36 to 18.81
 Apparent magnitude range for sigma=0.3: -44.36 to 19.45
 Apparent magnitude range for sigma=0.6: -44.64 to 19.95

(d) Write a python function that:

- reads the resulting m values from item (c),
- removes values with apparent magnitudes larger than the detection threshold $m = 18.5$ mag,
- re-calculates the mean observed magnitude $\langle M_{\text{observed}} \rangle$ of the SN Ia from the actually detected objects for the same redshift range.
- returns the bias as a function of redshift. The bias in M can be calculated with:

$$|\Delta M| = |\langle M_{\text{observed}} \rangle - \langle M \rangle| \quad (3)$$

```

In [6]: def bias_vs_redshift(z_values, M_values, detection_threshold=18.5):
      """
      Calculate bias as a function of redshift by applying detection threshold

      Parameters:
      z_values: array of redshift values

```

```

M_values: array of absolute magnitude values
detection_threshold: apparent magnitude detection limit (default: 18.5)
"""

# Calculate distances and apparent magnitudes
d, m = distances_and_magnitudes(z_values, M_values)

# Apply detection threshold - keep only objects with m >= detection_thre
detected_mask = m <= detection_threshold

# Calculate mean of original absolute magnitudes
M_mean = np.mean(M_values)

# Calculate observed mean absolute magnitude from detected objects
if np.any(detected_mask):
    M_observed_mean = np.mean(M_values[detected_mask])
else:
    M_observed_mean = np.nan # No objects detected

# Calculate bias
bias = np.abs(M_observed_mean - M_mean)

return z_values, bias, detected_mask, m

# Calculate bias for each sigma case
z1, bias1, mask1, m1_full = bias_vs_redshift(z_range, sigma_1)
z2, bias2, mask2, m2_full = bias_vs_redshift(z_range, sigma_2)
z3, bias3, mask3, m3_full = bias_vs_redshift(z_range, sigma_3)

print(f"Bias for  $\sigma=0.1$ : {bias1:.6f} mag")
print(f"Bias for  $\sigma=0.3$ : {bias2:.6f} mag")
print(f"Bias for  $\sigma=0.6$ : {bias3:.6f} mag")

print(f"Detected objects for  $\sigma=0.1$ : {np.sum(mask1)}/{len(mask1)}")
print(f"Detected objects for  $\sigma=0.3$ : {np.sum(mask2)}/{len(mask2)}")
print(f"Detected objects for  $\sigma=0.6$ : {np.sum(mask3)}/{len(mask3)}")

Bias for  $\sigma=0.1$ : 0.005664 mag
Bias for  $\sigma=0.3$ : 0.027980 mag
Bias for  $\sigma=0.6$ : 0.085744 mag
Detected objects for  $\sigma=0.1$ : 923/1000
Detected objects for  $\sigma=0.3$ : 910/1000
Detected objects for  $\sigma=0.6$ : 884/1000

```

(e) Make 3 plots of m versus N , where N is the number of generated objects, one for each $\sigma_M = 0.1, 0.3$, and 0.6 mag, showing the detection threshold and colouring distinctly the objects that would not be detected.

(f) Make 3 plots of $|\Delta M|$ versus z , one for each $\sigma_M = 0.1, 0.3$, and 0.6 mag.

(g) Based on your results, at which redshift does selection bias become important in each case?


```

In [7]: # Calculate apparent magnitudes for each sigma case
✓ d1_full, m1_full = distances_and_magnitudes(z_range, sigma_1)
✓ d2_full, m2_full = distances_and_magnitudes(z_range, sigma_2)
✓ d3_full, m3_full = distances_and_magnitudes(z_range, sigma_3)

# Detection threshold
✓ detection_threshold = 18.5

# Create plots for each sigma case
✓ fig, axes = plt.subplots(1, 3, figsize=(18, 6))

# Plot for sigma 1
detected_1 = m1_full <= detection_threshold
✓ axes[0].scatter(range(1, n+1), m1_full, c=['blue' if detected else 'red' for
      alpha=0.6, s=10, label=None)
# Add legend handles manually
✓ axes[0].scatter([], [], c='blue', alpha=0.6, s=30, label='Detected')
axes[0].scatter([], [], c='red', alpha=0.6, s=30, label='Not detected')
axes[0].axhline(y=detection_threshold, color='black', linestyle='--', linewidth=1,
      label=f'Detection threshold (m = {detection_threshold})')
axes[0].set_title(f'Apparent Magnitude vs N ( $\sigma$  = {sigmas[0]})')
axes[0].set_xlabel('Number of Objects (N)')
axes[0].set_ylabel('Apparent Magnitude m')
✓ axes[0].set_ylim(15, 22)
axes[0].grid(True, alpha=0.3)
axes[0].legend()

# Plot for sigma 2
detected_2 = m2_full <= detection_threshold
✓ axes[1].scatter(range(1, n+1), m2_full, c=['blue' if detected else 'red' for
      alpha=0.6, s=10, label=None)
# Add legend handles manually
✓ axes[1].scatter([], [], c='blue', alpha=0.6, s=30, label='Detected')
axes[1].scatter([], [], c='red', alpha=0.6, s=30, label='Not detected')
axes[1].axhline(y=detection_threshold, color='black', linestyle='--', linewidth=1,
      label=f'Detection threshold (m = {detection_threshold})')
axes[1].set_title(f'Apparent Magnitude vs N ( $\sigma$  = {sigmas[1]})')
axes[1].set_xlabel('Number of Objects (N)')
axes[1].set_ylabel('Apparent Magnitude m')
✓ axes[1].set_ylim(15, 22)
axes[1].grid(True, alpha=0.3)
axes[1].legend()

# Plot for sigma 3
detected_3 = m3_full <= detection_threshold
✓ axes[2].scatter(range(1, n+1), m3_full, c=['blue' if detected else 'red' for
      alpha=0.6, s=10, label=None)
# Add legend handles manually
✓ axes[2].scatter([], [], c='blue', alpha=0.6, s=30, label='Detected')
axes[2].scatter([], [], c='red', alpha=0.6, s=30, label='Not detected')
axes[2].axhline(y=detection_threshold, color='black', linestyle='--', linewidth=1,
      label=f'Detection threshold (m = {detection_threshold})')
axes[2].set_title(f'Apparent Magnitude vs N ( $\sigma$  = {sigmas[2]})')
axes[2].set_xlabel('Number of Objects (N)')
✓ axes[2].set_ylabel('Apparent Magnitude m')

```

✓

```
axes[2].set_ylim(15, 22)
axes[2].grid(True, alpha=0.3)
axes[2].legend()
```

```
plt.tight_layout()
plt.show()
```

```
# Print detection statistics
```

```
print(f"Detection statistics:")
```

```
print(f"σ = {sigmas[0]}: {np.sum(detected_1)}/{n} objects detected ({100*np.
```

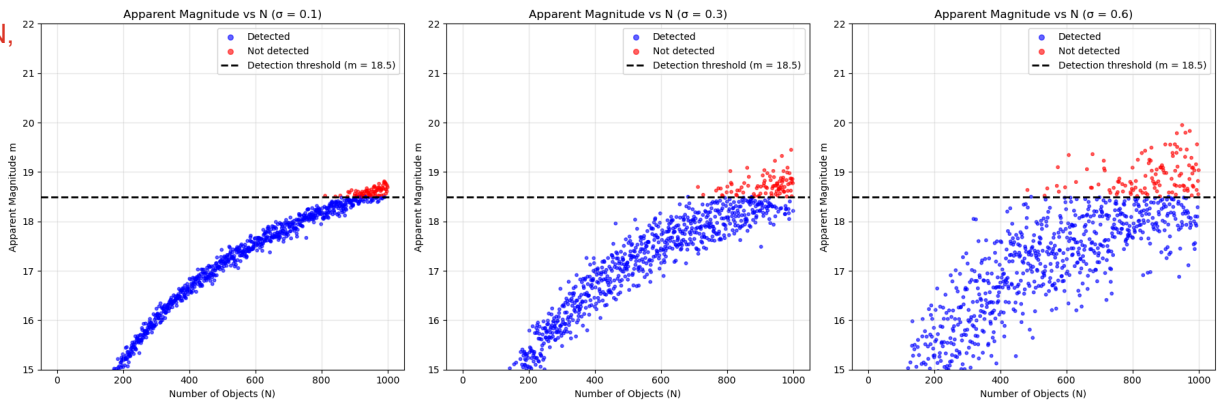
```
print(f"σ = {sigmas[1]}: {np.sum(detected_2)}/{n} objects detected ({100*np.
```

```
print(f"σ = {sigmas[2]}: {np.sum(detected_3)}/{n} objects detected ({100*np.
```

The points should
not be ordered by N,
but Ok.

✗

-0.5



Detection statistics:

✓

σ = 0.1: 923/1000 objects detected (92.3%)

σ = 0.3: 910/1000 objects detected (91.0%)

σ = 0.6: 884/1000 objects detected (88.4%)

In [12]: `def calculate_bias_vs_redshift(z_values, M_values, detection_threshold=18.5)`

```
.....
```

```
    Calculate bias as a function of redshift by applying detection threshold
```

```
.....
```

```
    biases = []
```

```
    z_vals = []
```

```
    for z in z_values:
```

```
        # Calculate distances and apparent magnitudes for this redshift
```

```
        d, m = distances_and_magnitudes(np.array([z]), M_values)
```

```
        # Apply detection threshold
```

```
        detected_mask = m <= detection_threshold
```

```
        if np.any(detected_mask):
```

```
            # Calculate bias for detected objects
```

```
            M_mean = np.mean(M_values)
```

```
            M_observed_mean = np.mean(M_values[detected_mask])
```

```
            bias = np.abs(M_observed_mean - M_mean)
```

```
        else:
```

```
            # No objects detected at this redshift
```

```
            bias = np.nan
```

```
        biases.append(bias)
```

```
        z_vals.append(z)
```

✓

✓

✓

✓

✓

```

    return np.array(z_vals), np.array(biases)

# Calculate bias vs redshift for each sigma case
z_bias_1, bias_z_1 = calculate_bias_vs_redshift(z_range, sigma_1)
z_bias_2, bias_z_2 = calculate_bias_vs_redshift(z_range, sigma_2)
z_bias_3, bias_z_3 = calculate_bias_vs_redshift(z_range, sigma_3)

# Create plots of  $|\Delta M|$  vs z
fig, axes = plt.subplots(3, 1, figsize=(10, 8))

# Plot for sigma 1
valid_1 = ~np.isnan(bias_z_1)
axes[0].plot(z_bias_1[valid_1], bias_z_1[valid_1], 'b-', linewidth=2, label=
axes[0].set_title(f'$|\Delta M|$ vs Redshift ($\sigma$ = {sigmas[0]})')
axes[0].set_xlabel('Redshift $z$')
axes[0].set_ylabel('$|\Delta M|$ (mag)')
axes[0].grid(True, alpha=0.3)
axes[0].set_xlim(0, 0.1)
axes[0].legend()

# Plot for sigma 2
valid_2 = ~np.isnan(bias_z_2)
axes[1].plot(z_bias_2[valid_2], bias_z_2[valid_2], 'g-', linewidth=2, label=
axes[1].set_title(f'$|\Delta M|$ vs Redshift ($\sigma$ = {sigmas[1]})')
axes[1].set_xlabel('Redshift $z$')
axes[1].set_ylabel('$|\Delta M|$ (mag)')
axes[1].grid(True, alpha=0.3)
axes[1].set_xlim(0, 0.1)
axes[1].legend()

# Plot for sigma 3
valid_3 = ~np.isnan(bias_z_3)
axes[2].plot(z_bias_3[valid_3], bias_z_3[valid_3], 'r-', linewidth=2, label=
axes[2].set_title(f'$|\Delta M|$ vs Redshift ($\sigma$ = {sigmas[2]})')
axes[2].set_xlabel('Redshift $z$')
axes[2].set_ylabel('$|\Delta M|$ (mag)')
axes[2].grid(True, alpha=0.3)
axes[2].set_xlim(0, 0.1)
axes[2].legend()

plt.tight_layout()
plt.show()

# Print analysis of when selection bias becomes important
print("Analysis of selection bias:")
print(f"sigma = {sigmas[0]}: Maximum bias = {np.nanmax(bias_z_1):.4f} mag")
print(f"sigma = {sigmas[1]}: Maximum bias = {np.nanmax(bias_z_2):.4f} mag")
print(f"sigma = {sigmas[2]}: Maximum bias = {np.nanmax(bias_z_3):.4f} mag")

print("-----")

# Find redshift where bias becomes significant (>0.01 mag)
threshold_bias = 0.01
for i, (sigma, bias_array, z_array) in enumerate([(sigmas[0], bias_z_1, z_bi
                                                    (sigmas[1], bias_z_2, z_bi
                                                    (sigmas[2], bias_z_3, z_bi

```

Ok.

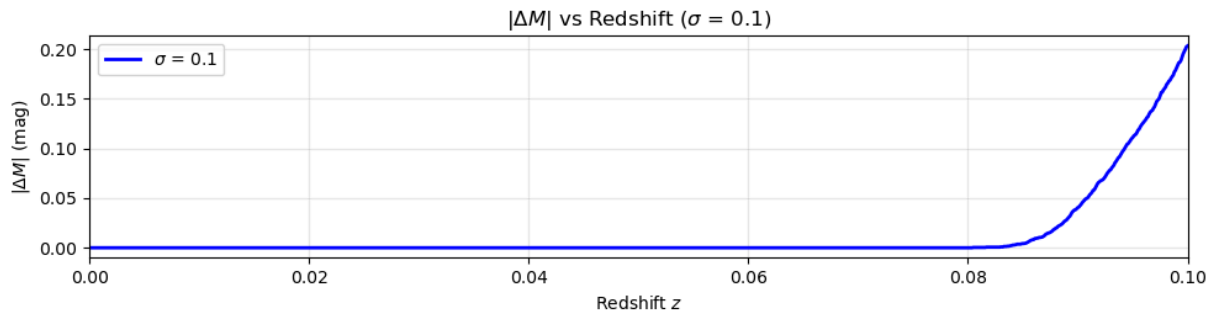
✓

```

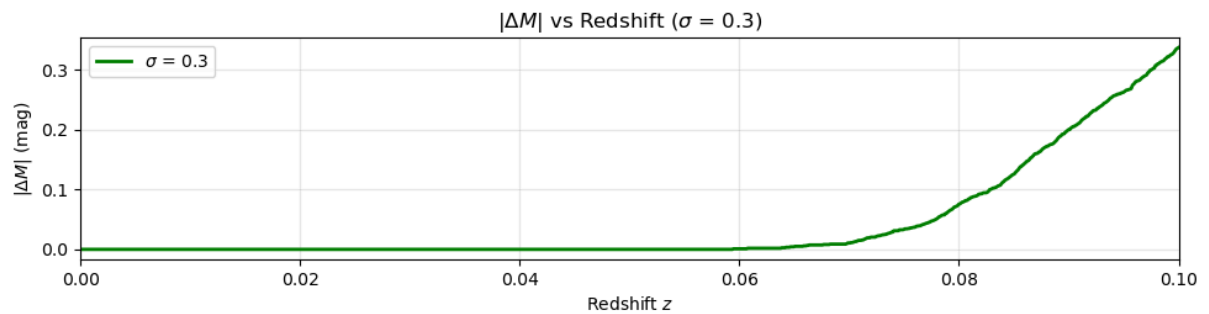
significant_bias = bias_array > threshold_bias
if np.any(significant_bias):
    first_significant = np.where(significant_bias)[0][0]
    print(f"sigma = {sigma}: Selection bias becomes important (>0.01 mag
else:
    print(f"sigma = {sigma}: Selection bias remains below 0.01 mag in th

```

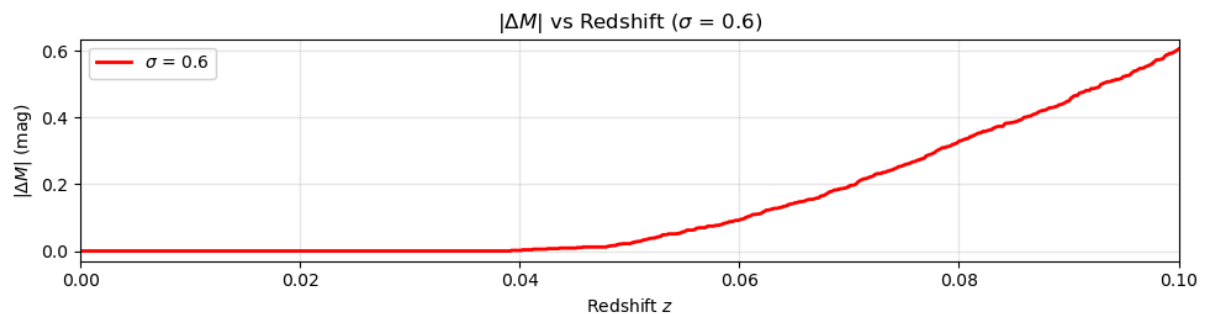
✓



✓



✓



✓

Analysis of selection bias:

✓

sigma = 0.1: Maximum bias = 0.2031 mag

sigma = 0.3: Maximum bias = 0.3375 mag

sigma = 0.6: Maximum bias = 0.6055 mag

sigma = 0.1: Selection bias becomes important (>0.01 mag) at $z \approx 0.086$

✓

sigma = 0.3: Selection bias becomes important (>0.01 mag) at $z \approx 0.070$

sigma = 0.6: Selection bias becomes important (>0.01 mag) at $z \approx 0.045$